Thesis Proposal
On Embedding Database Management System Logic in Operating Systems via Restricted Programming Environments

Matthew Butrovich

August 2023

Computer Science Department
School of Computer Science
Carnegie Mellon University
Pittsburgh, PA 15213

Thesis Committee:
Andrew Pavlo, Chair
Jignesh M. Patel
Justine Sherry
Samuel Madden (Massachusetts Institute of Technology)

Submitted in partial fulfillment of the requirements for the degree of Doctor of Philosophy.

Copyright © 2023 Matthew Butrovich

Abstract

The rise in computer storage and network performance means that disk I/O and network communication are often no longer bottlenecks in database management systems (DBMSs). Instead, the overheads associated with operating system (OS) services (e.g., system calls, thread scheduling, and data movement from kernel-space) limit query processing responsiveness. To avoid these overheads, user-space applications prioritizing performance over simplicity can elide these software layers with a kernel-bypass design. However, extracting benefits from kernel-bypass frameworks is challenging, and the libraries are incompatible with standard deployment and debugging tools. For these reasons, few DBMSs employ a kernel-bypass approach.

This proposal presents user-bypass—an approach to designing DBMS software that complements OS extensibility. With user-bypass, developers write safe, event-driven programs to push DBMS logic into the kernel's stack and avoid user-space overheads. We demonstrate user-bypass in the context of two different applications for DBMSs. First, we present TScout, a framework for training data collection in self-driving DBMSs. User-bypass accelerates TScout's metrics collection by not requiring multiple round trips to kernel-space to retrieve performance counters and other resource counters. Then, we present Tigger, a PostgreSQL-compatible DBMS proxy similar to RDS Proxy, PgBouncer, and ProxySQL. Through user-bypass, Tigger supports features like connection pooling, transaction multiplexing, and workload mirroring without user-space interaction.

We propose to extend our preliminary work by building a DBMS that executes queries with user-bypass. We will investigate the opportunities and limitations of placing core DBMS components in kernel-space. First, we will design and evaluate a storage manager that stores database entries in kernel-resident data structures, eliding the need to go to user-space to execute queries. Second, we will investigate concurrency control methods to enforce ACID properties within the constraints of kernel execution (e.g., no waiting). Lastly, we will create a framework for logging and checkpointing the database contents stored in kernel-space. This effort will fulfill the goal of crash recovery and inform the design of further uses for logging, like replication.

Contents

1 Introduction **1**

2 Background **3**
2.1 Kernel-Bypass vs. User-Bypass . . . 3
2.2 eBPF . . . 4

3 User-Bypass for Self-Driving DBMSs **7**
3.1 Self-Driving DBMSs . . . 7
3.2 Training Data . . . 8
3.2.1 Input Features Collection . . . 8
3.2.2 Output Metrics Collection . . . 8
3.3 Experimental Evaluation . . . 10
3.3.1 Workloads . . . 10
3.3.2 Runtime Overhead . . . 11

4 User-Bypass for DBMS Proxies **13**
4.1 DBMS Proxies . . . 13
4.2 Experimental Evaluation . . . 14
4.2.1 Workloads . . . 15
4.2.2 Connection Pooling for Many Clients . . . 15

5 Proposed Work **17**
5.1 Storage Management . . . 17
5.2 Transaction Management . . . 18
5.3 Logging and Checkpointing . . . 19

6 Thesis Timeline **21**

Bibliography **23**

Chapter 1

Introduction

Software applications rely on operating system (OS) services to manage hardware resources like CPU, memory, and I/O (e.g., network, disk). For decades, researchers discussed the shortcomings of using OS services to support database management systems (DBMSs). For example, memory management and I/O interfaces make it difficult to reason about durability, while thread scheduling and file system abstractions are deemed too slow [58]. With a monolithic OS kernel (e.g., Linux, UNIX), services are general purpose frameworks that applications interface with via system calls. Due to the large number of features they must support, kernel developers do not optimize these system calls for any one use case.

To improve performance, DBMSs employ domain-specific knowledge to manage system resources at the application level in user-space. For example, coroutines are a cooperative multithreading technique that elide the preemptive multitasking interface of the OS. This design can reduce the overhead associated with CPU scheduling, freeing resources for the DBMS to process queries more efficiently [37]. Similarly, to maximize inter-query parallelism concurrency and minimize cache misses, DBMSs manage the memory for database pages via buffer pools with custom eviction algorithms. These algorithms adapt to DBMS operations to prevent degenerate cases like sequential flooding. Techniques for the DBMS to manage disk and network resources in user-space are similarly complex to those for CPU and memory described above.

User-space applications prioritizing performance can elide OS services for I/O with a kernel-bypass design. In this approach, a user-space application communicates directly with the device driver—bypassing the kernel's stack. Thus, the application manages device and communication protocols and their associated state machines. Despite this added user-space logic, kernel-bypass aims to improve system performance by hoisting data to be processed in user-space as quickly as possible. However, achieving good performance from kernel-bypass frameworks is challenging [64], and the libraries are incompatible with standard deployment and debugging tools [60].

The techniques described above are all-or-nothing approaches, whereby the DBMS wrests complete control of a resource from the OS. As a compromise between complete reliance on the OS and techniques like kernel-bypass, the research community proposed methods to extend OS behavior to better suit user-space applications [25, 53, 54]. However, these approaches lacked a standard API and strong OS support to extend behavior. Until recently, if a developer wanted

to embed DBMS logic into the kernel, they would have to either (1) load a kernel module or (2) modify and recompile the kernel. Such approaches are notoriously difficult and sacrifice system reliability and safety.

This thesis seeks to address the challenge of designing DBMSs that complement OS extensibility. The technique we propose is *user-bypass*, whereby developers push DBMS logic into the kernel's stack to avoid copying data between user-space and kernel-space. The technology that enables user-bypass and addresses the shortcomings of past OS extensibility efforts is *extended Berkeley Packet Filter* (eBPF). This modern Linux subsystem enables developers to write safe, event-driven programs that run in kernel-space [56].

This thesis aims to provide evidence to support the following statement:

Thesis Statement: *Embedding database management system logic in the operating system via modern kernel extension mechanisms to bypass user-space for frequently performed operations is a safe and maintainable way to improve system performance and lower deployment costs.*

First, we provide an overview of how DBMSs interact with OS services in Chapter 2. Next, we discuss our completed frameworks in Chapters 3 and 4. Then, we introduce our proposed kernel-space DBMS in Chapter 5. Lastly, we present the thesis timeline in Chapter 6.

Chapter 2

Background

We first define our user-bypass method in the context of Linux networking and existing kernel-bypass techniques. We then discuss the kernel feature that enables user-bypass and its limitations.

2.1 Kernel-Bypass vs. User-Bypass

To discuss kernel-bypass and user-bypass, we first provide an overview of the relevant OS services for I/O. We detail the OS networking stack, but the storage stack contains similar software layers (e.g., block layer, virtual file system).

A complete overview of Linux's networking stack is beyond the scope of this chapter; we will focus on the portion relevant to DBMSs. Linux contains multiple layers for processing network traffic. These layers handle transport (e.g., TCP), network (e.g., IP), and link layer (e.g., Ethernet) protocols. The stack exposes a socket interface [20] for applications to copy data between user-space and kernel-space, along with traffic control [21] interfaces to configure queuing disciplines and network filters.

User-space applications pioritizing performance over simplicity can elide these software layers using *kernel-bypass* methods. In this scenario, a user-space application receives bytes directly from the device driver — bypassing the kernel's network stack. Thus, the application manages communication protocols and their associated state machines. The most common kernel-bypass pattern for network applications is to use Intel's Data Plane Development Kit (DPDK) [6] software library with user-space TCP implementations like mTCP [39]. Intel provides the similarly named Storage Performance Development Kit (SPDK) [22] for storage devices. Other kernel-bypass libraries (e.g., F-Stack [8] and ScyllaDB's Seastar [19]) attempt to simplify development by bundling DPDK with bespoke TCP logic. In 2018, Linux added native kernel-bypass support with AF_XDP [38], removing the dependency on out-of-tree kernel modules such as DPDK.

Historically, kernel-bypass was the preferred way to implement high-performance Linux networking applications. There are several reasons for this view: (1) the Linux networking stack was perceived as slow and inefficient, (2) applications performed encryption in user-space using a software library (e.g., GnuTLS, LibreSSL, OpenSSL), and (3) a lack of programmability in

the networking stack. The interfaces to program the network stack were limited to filtering and routing decisions based on L2/L3 rules (e.g., `iptables`, `nftables`, `tc`). This lack of extensibility in the kernel prevented L7 DPI necessary for a DBMS proxy. Thus, kernel-bypass remained viable for network applications with complex user-space logic.

To our knowledge, the only database vendor making significant use of kernel-bypass is Yellowbrick, which does not use DPDK but instead wrote their own network and storage device drivers [36]. Although ScyllaDB promotes DPDK compatibility in their Seastar framework, they do not deploy DPDK in production [35]. In general, deploying DPDK applications into production is difficult due to API/ABI instability, which forces developers to choose between features and fixes of new releases and code stability of LTS releases [46].

Kernel-bypass aims to improve performance by transferring buffers between devices and user-space applications instead of the kernel processing them. *User-bypass* is the opposite approach: the developer pushes application logic into the kernel's network stack as low as possible to avoid copying data between user-space and kernel-space, while benefiting from the kernel handling L1-L4 networking. Until recently, if a developer wanted to embed application logic into the kernel, they would have to either (1) load a kernel module or (2) modify and recompile it. Such approaches are difficult and sacrifice system reliability and safety. However, updates to Linux make user-bypass a viable alternative to kernel-bypass. The efficiency of the Linux networking stack has also improved: a single CPU core can process 42 Gbps [28], and dedicated servers can process 670 Gbps of data [23]. Next, kernel TLS (kTLS) allows developers to move encryption into kernel-space and hardware [55]. But the key reason that user-bypass is now possible is the increased programmability via eBPF, which is composable with kTLS [26].

2.2 eBPF

To understand user-bypass, we now provide an overview of how the technique embeds DBMS logic in the Linux kernel. The primary technology that enables this functionality is *extended Berkeley Packet Filter* (eBPF). This modern Linux subsystem enables developers to write safe, event-driven programs running in kernel-space [56]. Application developers and cloud vendors have rapidly adopted eBPF due to its safety and features [3, 4, 5]. For example, Meta loads over 40 eBPF programs on every server, with hundreds more loaded on demand [57].

eBPF programs run a limited instruction set in a kernel-embedded virtual machine. Developers typically write eBPF programs in higher-level languages like C or Rust that compile to eBPF bytecode via LLVM. Upon loading the eBPF program, modern kernels compile the bytecode to native machine code. eBPF program capabilities vary based on their type and attachment point, but they attach to predetermined functions in the OS stack for network processing.

Developers load eBPF programs into kernel-space and then associate them with events (e.g., functions, static tracepoints) to trigger their execution. When a running thread hits the attachment point, it starts the execution of the eBPF program in privileged mode. Depending on their behavior and attachment location, developers use eBPF programs for software debugging, profiling, or modifying data flow (e.g., network buffers) through kernel-space.

The execution state of eBPF programs is ephemeral, meaning that its decision-making is limited to the data available during a single invocation of the handler. However, with *eBPF*

maps, the program can maintain state across events, enabling user-bypass to support more complex application behavior. These data structures reside in kernel-space and are the primary mechanism for creating stateful user-bypass programs.

Because the CPU is in privileged mode when eBPF programs run, the kernel requires them to pass a verification step before it loads them. The eBPF verifier enforces kernel API compliance, memory access safety, execution bounds, and instruction count. The verifier generates a control flow graph for all possible branches of the eBPF program and checks limits like 512 B stack size and 1m instructions. Although these restrictions limit the complexity of application logic with user-bypass, DBMS network protocols for the most common message types (i.e., queries, results) are expressible in eBPF. Despite the verifier's guarantees, developers should consider security and safety practices with eBPF deployment due to its close interaction with kernel functions, especially in multi-tenant environments.

Chapter 3

User-Bypass for Self-Driving DBMSs

3.1 Self-Driving DBMSs

Self-driving DBMSs seek to automate the arduous tuning and optimization tasks for databases [13, 49, 50]. Given a target objective function (e.g., throughput, latency), a self-driving DBMS automatically deploys actions that it deems will help the application's future workload for that objective. These actions control three aspects of the system: (1) physical design, (2) knob configuration, and (3) hardware resources. Although there are existing advisory tools that support individual actions (e.g., index recommendation [32], knob tuning [61]), a self-driving DBMS takes a holistic approach to optimization to include when and how to deploy such actions.

A self-driving DBMS's architecture is comprised of three components: (1) forecasting system, (2) behavior models, and (3) planning system [50]. The *forecasting system* is how the DBMS observes and predicts the application's future workload [42]. The DBMS then uses these forecasts with its *behavior models* to predict its runtime behavior relative to the target objective (e.g., latency, throughput) [43]. The *planning system* selects actions that improve this objective.

A behavior model represents a discrete component in the DBMS. For a set of input features, the model emits metrics that estimate the component's work for those inputs. For example, a DBMS could build a model for a sequential scan where the input is the table name and the number of tuples that the operator will scan, and the output is the expected execution time.

The models' output is comprised of one or more *metrics*. A metric is a low-level measurement of how the DBMS interacts with its underlying hardware: (1) CPU, (2) memory, (3) disk, and (4) network. The DBMS can easily measure some metrics with user-level code (e.g., memory allocated). Other metrics are only observable via methods that are external to the DBMS in either the OS or hardware (e.g., CPU counters), and thus are more challenging to collect. The self-driving components require accurate models. For example, the planning system uses models and the workload forecast to predict future DBMS resource consumption. If those models are inaccurate, the planning components will not optimize the DBMS.

The scope of a model depends on the implementation. In the case of the sequential scan example, the DBMS could use a single model to represent the entire query plan scan operator [44] or multiple models with internal features that represent smaller operating units (OUs). OU-level models are less complex and thus require less training data than monolithic models [43].

3.2 Training Data

To build OU behavior models, a self-driving DBMS needs training data of previous examples of the system's operations (e.g., executing queries). Each data point in a training corpus contains input features and its corresponding output metrics that the models will predict. We now describe ways in which a DBMS can generate such training data.

3.2.1 Input Features Collection

The inputs to an OU behavior model describe the DBMS's task when executing that OU. For example, the features for an index lookup OU contain its schema, data structure type, and the number of index entries. Since workloads, statistics, and configurations fluctuate, such features must reflect DBMS state at the time of execution. One can collect these features from the DBMS either (1) externally via SQL commands or (2) internally within the DBMS.

External: Previous work on modeling query latency extracts features by executing `EXPLAIN` for every query [44]. This provides the optimizer's physical plan and cost estimates, which can be decomposed into individual operator features.

However, this approach is not feasible in high-performance scenarios. `EXPLAIN` is meant to be an infrequent operation that regenerates the query plan. Furthermore, it presents human-readable information about query execution, and cannot capture a query's interactions with DBMS background tasks, like garbage collection and write-ahead logging (WAL). Lastly extracting the DBMS's configuration and environment requires executing even more SQL queries. This additional work slows down query execution, making it challenging to collect training data in an online setting.

Internal: An alternative approach is to embed feature collection logic inside the DBMS. Since the DBMS already maintains information about query execution, configuration, and environment, it can use its internal APIs to read this data. In addition, the system can tag background operations with identifiers to describe what queries initiated them. This approach is similar to previous network tracing methods that determine causality in opaque systems [34].

Internal methods solve many problems associated with external training data features at the expense of higher software engineering costs. Foremost is that since the DBMS already needs the results of `EXPLAIN` for query execution—explicitly invoking it is not necessary. Second, internal collection accurately represents temporal features that fluctuate, like CPU frequency and the number of concurrent workers. Lastly, internal feature collection enables modeling DBMS subsystems beyond the query execution to express causality in background tasks like write-ahead logging. As such, an internal collection method is better suited for self-driving DBMSs.

3.2.2 Output Metrics Collection

The output of an OU behavior model is the DBMS's estimated metrics, so the training data for these models must come from runtime operations. Therefore, the challenge is how to efficiently

measure a DBMS's metrics without altering its observed behavior.

Previous work on modeling query latency collected metrics using `EXPLAIN ANALYZE` [44]. In addition to `EXPLAIN`'s limitations that we describe in Section 3.2.1, `EXPLAIN ANALYZE` does not return query results on the client, and the instrumentation imposes runtime overhead to query execution [16]. It can also not collect metrics for other parts of the system not related to query execution (e.g., maintenance operations). Such issues make `EXPLAIN ANALYZE` unsuitable for generating the output metrics for training data.

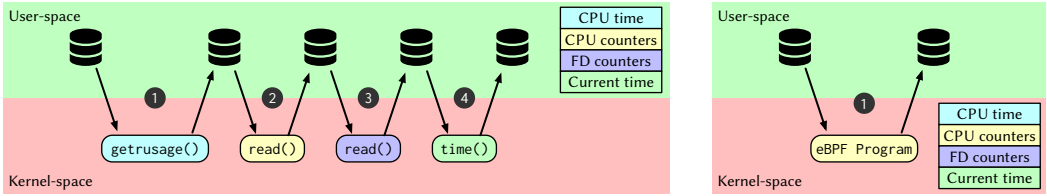
Given this, there are two approaches for capturing metrics in an internal training data collection framework for a self-driving DBMS: (1) user-space or (2) user-bypass.

User-space: To collect system metrics from user-space, the DBMS manually invokes the necessary functions in its source code to enable and disable recording metrics. It then manually retrieves the metrics with a final set of function calls.

The problem with this approach is that for some metrics, such as CPU counters, the functions to enable/disable their collection are syscalls to the OS; syscalls are expensive because execution switches into a privileged kernel mode. In some cases, collecting metrics for a single OU requires multiple syscalls per hardware type. Another problem is that the OS may not expose all the metrics via syscalls. Instead, the DBMS developer has to scrape procfs (`/proc`) or other tools, which are slower and less robust than syscalls.

User-Bypass: With user-bypass collection, the DBMS relies on ancillary eBPF programs running inside the OS kernel to retrieve metrics for it. This approach means that the program requires only one transition to kernel-space to extract multiple system metrics. Once in kernel-space, the program can read CPU performance counters, network statistics, and any additional system metrics.

Fig. 3.1 shows how program execution transitions between user-space and kernel-space when collecting metrics with different approaches.



(a) User-space **(b)** User-bypass

Figure 3.1: Metrics Collection Techniques – DBMSs typically collect runtime metrics via multiple systems calls and then aggregate the results in user-space as shown in Fig. 3.1a. User-bypass uses eBPF programs to (1) collect all desired metrics with a single transition to kernel-space and (2) aggregate results in kernel-space as shown in Fig. 3.1b.

3.3 Experimental Evaluation

We implemented a user-bypass training data collection framework called **TScout** in the NoisePage self-driving DBMS. TScout collects metrics that are a combination of hardware-level performance counters, kernel-level observations, and application-level counters. TScout combines these metrics with descriptors of DBMS behavior during query execution and internal maintenance tasks to create training data for machine learning models. TScout uses code generation to create a eBPF program that collects metrics tailored to the DBMS.

NoisePage is a PostgreSQL-compatible DBMS that uses HyPer-style MVCC [48] over Apache Arrow in-memory columnar data [41]. The original version of NoisePage uses OU-level behavior models that it trains via offline runners with user-space metrics collections [43]. We modified NoisePage's source code to introduce TScout's user-bypass framework for its OUs.

We deployed NoisePage on a server with 2×20-core Intel Xeon Gold 5218R CPUs, 196 GB DRAM, Samsung PM983 SSD, and Ubuntu 20.04 with Linux (v5.4) that supports eBPF.

3.3.1 Workloads

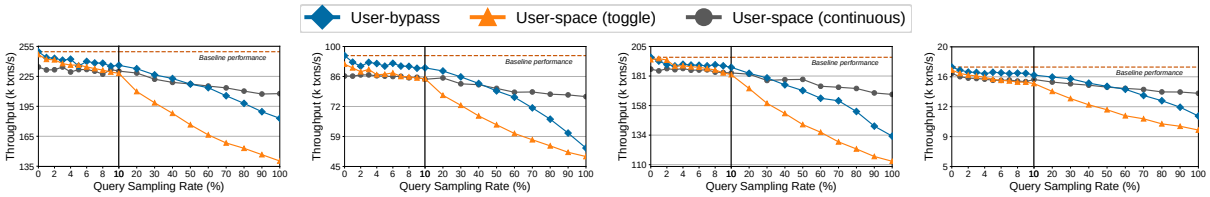
We use the following workloads in our evaluation. All queries execute over JDBC using the BenchBase framework [2, 33].

YCSB: The Yahoo! Cloud Serving Benchmark [31] is a synthetic benchmark modeled after cloud services. To maximize the transaction throughput, we use a read-only configuration for YCSB that only executes queries that retrieve a single tuple using its primary key. The YCSB database contains a single table comprised of tuples with a primary key and 10 columns of random data, each 100 bytes in size. Each tuple's size is approximately 1 KB. We use a database with 12m tuples (~13 GB).

SmallBank: This workload models a banking application where transactions perform simple read and update operations on customers' accounts [24, 27]. All transactions involve a small number of tuples retrieved using primary key indexes. In addition to the original six transaction types, we added a transaction that transfers money between two accounts. The database contains three tables with 50m accounts (~10 GB).

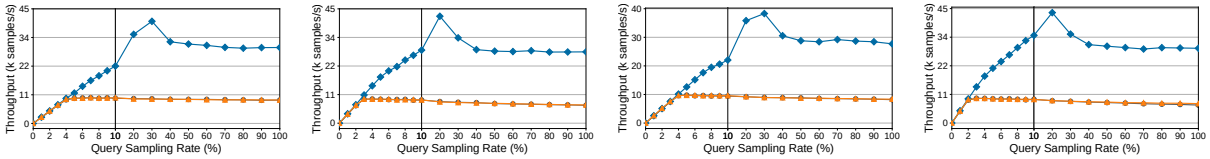
TATP: The Telecom Application Transaction Processing benchmark is an OLTP testing application that simulates a caller location system used by telecommunication providers [62]. It has nine transaction types that use either a primary key to find caller records or a secondary index as an indirection look-up to caller records. The database contains 20m tuples (~16 GB) stored across four tables.

TPC-C: This is an order-processing application that contains nine tables and five transaction types [59]. For our experiments, we use a 200-warehouse database (~25 GB).



(a) YCSB **(b)** SmallBank **(c)** TATP **(d)** TPC-C (200-warehouse)

Figure 3.2: Runtime Overhead (Transaction Throughput) – Impact of query sampling on OLTP transaction throughput, comparing user-space and kernel-space approaches to system metrics collection.



(a) YCSB **(b)** SmallBank **(c)** TATP **(d)** TPC-C (200-warehouse)

Figure 3.3: Runtime Overhead (Training Data Generation) – Impact of query sampling on OLTP training data generation, comparing user-space and user-bypass approaches to system metrics collection.

3.3.2 Runtime Overhead

We first measure to what extent collecting training data with TScout reduces the DBMS's performance. We modified NoisePage to use three collection methods with TScout: (1) user-bypass metrics collection with continuous `perf` counters (user-bypass), (2) user-space metrics collection with dynamic `perf` counters (user-space *toggle*), and (3) user-space metrics collection with continuous `perf` counters (user-space *continuous*). For the first and last methods, TScout enables OS `perf` counters even if the framework never retrieves the data (e.g., TScout toggles on CPU counters for all OUs at DBMS start-up). The second approach dynamically toggles `perf` at the start and stop of each OU.

This experiment enables data collection for all DBMS subsystems to ensure the maximum impact. We sweep the TScout's sampling rate from 0% to 100%. We run each rate configuration three times and report (1) the average throughput and (2) how many data samples TScout generates. We execute the workloads described in Section 3.3.1 with 20 client threads. We use user-bypass with 0% sampling rate as the baseline because the only user-space logic is minimized to sampling behavior, which all three methods need.

The results in Fig. 3.2 show that the DBMS's performance drops as the sampling rate increases for each method. This reduction is because the DBMS, OS, and TScout spend more time collecting and processing training data as the rate increases. In the user-space *toggle* configuration when the sampling rate reaches 100%, the DBMS's throughput drops by almost 50%. The user-space *continuous* approach reduces the DBMS's throughput by 2–8% even when the sampling

rate is 0% because the kernel does more work at each context switch to save the CPU's PMU state. But since user-space *continuous* only requires a single syscall to retrieve `perf` counters, it incurs at most a 15% reduction compared to the other methods. user-bypass requires multiple syscalls to read `perf` counters, which slow the system down despite the syscalls occurring under a single switch in execution mode. user-space *toggle* is the slowest because it requires three syscalls with execution mode switches for each sampled event: one to enable counters, disable counters, and read results.

user-space *continuous* has the smallest impact on the DBMS's performance at a high sampling rate, but we must also consider the data generation rate. As such, we next compare the number of data points that TScout extracts from the DBMS as the sampling rate increases. We measure this from the number of training data samples that TScout writes to its output source.

Fig. 3.3 shows TScout's data generation throughput increases as the sampling rate increases but only up to a point. For user-space *toggle* and user-space *continuous*, the overhead of retrieving data from the user-space probes prevents them from generating more data points beyond a 2–4% sampling rate. On the other hand, TScout generates data using user-bypass at a 3× higher generation rate than the other two methods. user-bypass achieves peak throughput at a 20–30% sampling; at a higher sampling rate, the overhead of collecting the data from kernel counters slows down query execution, leading to less collected data. We attribute user-bypass's advantage to its in-kernel data structures with efficient RCU synchronization [45] and reduced round trips to kernel-space.

The results in Figs. 3.2 and 3.3 show that TScout's user-bypass implementation with a sampling rate of 10% yields the maximum throughput for both the application's workload and training data generation.

Chapter 4

User-Bypass for DBMS Proxies

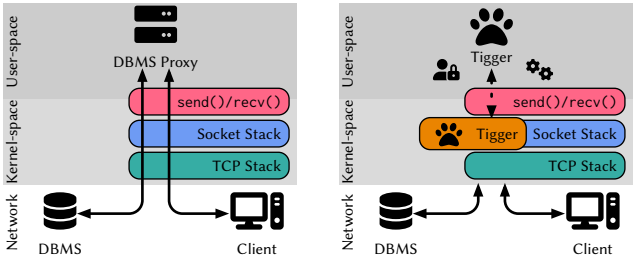
4.1 DBMS Proxies

Modern cloud applications often connect to DBMSs over a network in a manner that is not optimal for performance: either maintaining large numbers of persistent, mostly idle connections or rapidly churning connections. Both scenarios cause the DBMS to squander resources on state information for each connection or perform onerous and redundant network operations. One standard solution to overcome this problem is to use a *proxy* that acts as a middlebox [29] between the front-end client and back-end DBMS. Examples include AWS RDS Proxy [18], PgBouncer [14], Pgpool-II [15], MaxScale [10], and ProxySQL [17]. Although their features vary, these proxies all implement a target DBMS's network protocol so that clients can connect to them without needing to modify application code.

Using a DBMS proxy as the authoritative connection manager for the back-end DBMS provides multiple benefits. The proxy maintains a persistent connection pool to the back-end DBMS to multiplex client requests and responses—minimizing the number of connections talking to the DBMS. This optimization reduces the overhead of tasks that scale linearly with the number of clients like transaction commit. Furthermore, because the proxy's pooled connections only authenticate with the DBMS once, it prevents the DBMS from repeatedly performing connection setup procedures and frees system resources for query execution.

DBMS proxies contain logic specific to a back-end DBMS's network protocol, thus making them L7 proxies. This behavior contrasts with L3/L4 proxies (e.g., HAProxy [9], nginx [11]) that transparently perform load balancing at lower layer protocols. Such L7 logic requires processing network data in user-space (i.e., above the OS networking stack) or applying deep packet inspection (DPI) in lower levels of the network stack.

To the best of our knowledge, all existing DBMS proxies follow the same design in Fig. 4.1a: they are event-driven user-space applications that, after the authentication steps, (1) read client messages from a network socket, (2) inspect the stream of bytes, (3) match the client to a back-end server, and (4) send the data on the matched socket. Query results follow a similar logic but with the sender and receiver reversed. Our inspection of open-source proxies supports this belief, though implementation details vary from proxy to proxy. PgBouncer is written in C and uses the libevent library for notifications but is entirely single-threaded. Odyssey [12] is also



(a) User-space proxy **(b)** User-bypass proxy

Figure 4.1: User-Space and User-Bypass DBMS Proxies – User-space proxies rely on system calls to redirect queries and results between clients and DBMSs. Tigger employs user-bypass as a fast path in kernel-space, only passing authentication and user settings message to user-space.

in C, directly calls `epoll` for event management, but uses a bespoke coroutine library written in assembly and asynchronous IO to enable parallelism. ProxySQL is in C++, relies on the `poll` system call, and follows a more typical multi-threaded design.

These proxy implementations coordinate `send()` and `recv()` system calls with DBMS protocol-specific control logic. This design limits their scalability as network bandwidths increase. Research shows that copying buffers during system calls accounts for ~50% of the kernel's network stack CPU cycles [28]. PostgreSQL developers concluded that a saturated PgBouncer process spends most of its time copying data in and out of buffers between user-space and kernel-space [30]. Ideally, a DBMS proxy should do most of its work without copying socket buffers to user-space via system calls.

4.2 Experimental Evaluation

To demonstrate the effectiveness of user-bypass, we implemented a PostgreSQL-compatible proxy called **Tigger** using user-bypass. As shown in Fig. 4.1b, Tigger embeds DBMS protocol logic in kernel-space to achieve user-bypass for queries and their results. It offers efficient connection pooling and workload mirroring within kernel-space. Tigger supports the target DBMS's network protocol and is a drop-in replacement for other proxies.

We evaluate Tigger using PostgreSQL (v14.5) DBMS, and configure the `shared_buffers` knob so that working sets fit in memory. We compare Tigger against three other open-source PostgreSQL-compatible proxies:

- **PgBouncer (v1.17):** With decades of development, PgBouncer is the most popular proxy for connection pooling with PostgreSQL. Due to its popularity and maturity, we use PgBouncer as the reference implementation for user-space DBMS proxies.
- **Odyssey (v1.3):** Yandex developed Odyssey as a modern replacement for PgBouncer. It uses multiple workers and coroutines to support parallelism across connections. We use Odyssey as an example of a high-performance user-space DBMS proxy.

Unless otherwise specified, all experiments run as follows. We evaluate Tigger using AWS EC2 c6i instances running Ubuntu Linux 22.04 LTS in the same availability zone [7]. We use

separate instances for each system component: (1) 12xlarge for PostgreSQL, (2) 12xlarge for application servers, and (3) and xlarge for DBMS proxies. We use a smaller instance for the proxies to reflect how users provision proxies in real-world deployments. To better compare user-bypass, we reduce the proxy server's number of receive queues to one to eliminate kernel parallelism for network processing [1].

Each experiment runs at least five times. Latency evaluations use a fixed submission rate of 2000 transactions per second (TPS). This setup ensures all systems perform the same work to compare latency and CPU efficiency. Throughput evaluations run with an unlimited submission rate.

4.2.1 Workloads

In these experiments, we only consider OLTP workloads; OLAP workloads (e.g., TPC-H) are bottlenecked by query execution at the DBMS (e.g., scans, aggregations), which makes them unsuitable for demonstrating the benefits of Tigger's user-bypass design. In a serverless OLAP scenario, connection establishment overhead will not be the bottleneck compared to query execution. Customers deploy DBMS proxies for OLAP for reasons other than performance (e.g., security) outside this evaluation's scope.

All queries execute over JDBC using the BenchBase framework [2, 33]. We turn off automatically prepared statements in the JDBC driver to avoid naming contamination when the back-end connections are shared across client connections. This problem occurs when drivers prepare different statements under the same name, so DBAs turn off this feature when deploying a DBMS proxy.

- **No-op:** The workload contains a single transaction that executes an empty query string (i.e., "; "). The DBMS returns an empty query result. This workload is ideal for measuring DBMS proxy overhead because it minimizes the work that the client and DBMS execute.
- **SmallBank:** This workload models a banking application where transactions perform short read and update operations [24, 27]. The database contains three tables which we scale to ~3.4 GB.
- **TATP:** This benchmark simulates a cellphone caller location system [62]. It has nine transaction types, and we scale the database to ~1.9 GB stored across four tables.
- **TPC-C:** This order-processing application contains nine tables and five transaction types [59]. For our experiments, we use a 20-warehouse database (~2.1 GB).
- **Twitter:** This workload models the troubled social media website where users post messages and follow others. We scale the database to ~4.4 GB.
- **YCSB:** The Yahoo! Cloud Serving Benchmark models cloud service workloads [31]. We run read-only transactions to reduce bottlenecks from EBS writes. The database contains a single table, 1 KB tuples, and a total database size of ~2.3 GB.

4.2.2 Connection Pooling for Many Clients

The workload represents a typical cloud-native application. We use 25 application servers, each with 400 clients, for 10,000 database connections. The DBMS utilization is low, with each of the

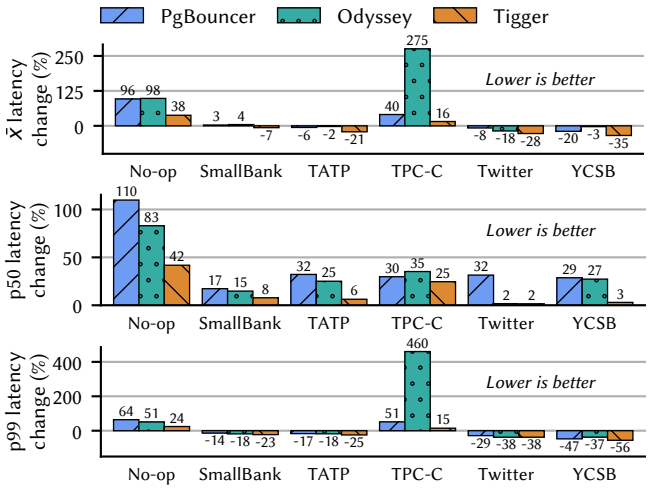


Figure 4.2: Connection Pooling for Many Clients – Percent change of comparing different DBMS proxies against connecting directly to PostgreSQL.

25 application servers submitting 80 TPS distributed across their connections, for a total of 2,000 TPS arriving at the back-end DBMS.

Fig. 4.2 shows the percent change for each DBMS proxy compared to the scenario with no proxy. Due to its user-bypass design, Tigger offers the lowest latencies of any DBMS proxy in every workload. Compared to running without a proxy, Tigger lowers the mean and p99 latencies in every workload except No-op and TPC-C.

No-op does not benefit from using a DBMS proxy. Even at a high connection count, PostgreSQL performs minimal work for No-op, so reducing the number of connections with a DBMS proxy provides no benefit. However, as a microbenchmark for proxies' protocol efficiency, we see that Tigger offers up to 31% lower latency than the other proxies. The size of the ingress query and egress results are essentially zero, so we are seeing the benefits of user-bypass eliding system calls and user-space thread scheduling.

Chapter 5

Proposed Work

In Chapters 3 and 4, we applied user-bypass both to collect training data for self-driving DBMSs and to manage connections between clients and DBMSs. These solutions demonstrate the benefit of user-bypass for tasks that are ancillary to DBMSs. However, user-bypass could also aid the core DBMS feature of query processing. For example, when a query arrives over the network at a typical DBMS, system calls copy the query contents to user-space. The DBMS performs the requested query operations on the database using more systems calls for I/O, synchronization, and memory management. Lastly, the DBMS sends the query results back to the client with a system call. User-bypass could elide these system calls, user-space thread scheduling, and associated data movement by performing query processing in kernel-space.

To this end, we propose building a DBMS that relies entirely on user-bypass for query processing. This DBMS must operate within the limits of eBPF's event-driven execution environment, restricted kernel API, and program verifier. We will focus on the fundamental ACID properties of user-bypass DBMSs by building an in-memory key-value store that supports multi-statement transactions and crash recovery. To leverage existing tooling and benchmarks, this DBMS will use the Redis network protocol. We will examine the core components of DBMSs, and investigate the benefits and challenges of redesigining them for user-bypass. In the following sections, we first discuss the design of a user-bypass storage manager. Then, we propose an approach for a user-bypass transaction manager. We finish with an overview of how a user-bypass DBMS will support durability and crash recovery.

5.1 Storage Management

DBMSs rely on storage managers to store, organize, and retrieve their database entries. This component builds upon complex data structures that ensure correctness even under concurrent operations. For example, a typical storage manager will store its databases in fixed-size pages that it organizes into pools with specialized eviction policies (e.g., LRU, clock). The storage manager is also responsible for maintaining any supplemental data structures for indexes like B+ trees or hash tables. Some storage manangers are key-value based, and rely entirely on tree-based data structures like the log-structured merge (LSM) tree. We will focus on an in-memory storage manager for variable-length key-value pairs, which generalizes to other data types like

SQL tables with variable schemas and data types.

Storing variable-length entries with minimal memory fragmentation and responsive performance is a challenge for user-space storage managers. Calling `malloc()` for each different entry is not performant, and workloads with a large distribution in entry sizes results in wasted memory due to fragmentation. A common solution is to use a small, fixed sized buffer for each entry. This buffer stores metadata (e.g., versioning information, locks), and either the inlined value or a prefix of the value and a pointer to a preallocated buffer on the heap that contains the full value [41]. Yet, as discussed in Chapter 2, eBPF programs cannot dynamically allocate memory. Thus, efficiently storing database values in kernel-space is the first major challenge that we must overcome.

We propose an iterative approach to building the storage manager. The first design will use a single eBPF hash map for the entire database, where the keys are `char[MAX_KEY_SIZE]` and the values are `char[MAX_VALUE_SIZE]`. We will choose reasonable values for these limits based on existing systems. However, this design trades simplicity for memory fragmentation. Every key-value pair will use the maximum number of bytes regardless of the actual size of the entry. This storage manager will serve as a baseline implementation as we build out and test other components of the system.

Our system will store the database in multiple eBPF maps, drawing inspiration from slab allocators. For example, a eBPF map with values of `char[16]` would store values up to 16 bytes, a eBPF map with values of `char[32]` would store values up to 32 bytes, etc. This approach would also require a top-level directory map—this time pointing to the correct slab to look in for a given key while also containing other metadata related to concurrency control (e.g., timestamps, locks).

The initial storage manager will not support MVCC, which may come later based on our explorations into concurrency control schemes. If the storage manager supports MVCC, we will also need to explore garbage collection (GC) techniques to clean up old versions. We believe that a eBPF program running at a fixed interval will be more performant than cooperative GC for several reasons. First, cooperative GC introduces more conditional logic to query execution, which makes it harder to satisfy the verifier. Short, single-purpose eBPF programs are more likely to pass verification. Second, eBPF programs are easily scheduled at fixed execution intervals due to their prevalence as sampling-based observability tools. The need for MVCC will depend on our results from the work proposed in Section 5.2.

5.2 Transaction Management

Multi-statement transactions is a hallmark feature for DBMSs. The transaction manager applies policies to enforce a concurrency control protocol. These protocols maintain the ACID properties of the DBMS, ensuring (1) all operations in a transaction appear to occur atomically, (2) the database maintains a consistent view to users, (3) transactions run in isolation from each other, and (4) and changes to the database are durable before a transaction completes.

Transaction managers apply concurrency control methods like two-phase locking (2PL), optimistic concurrency control (OCC), or timestamp ordering (TO) to allow for inter-query parallelism [63]. Each method has trade-offs, and multi-versioned systems incur additional

choices like version chain ordering (e.g., newest-to-oldest, oldest-to-newest), garbage collection schemes, and version contents.

Initially, we will support the Redis protocol's `MULTI` command that buffers subsequent statements until they all run atomically with `EXEC`. The first implementation will guarantee the DBMS executes transactions in serial order. By using a single kernel thread to handle DBMS requests, along with transactions that do not perform any work until `EXEC`, we can simplify the design of atomic operations within the DBMS. Yet, this design will limit concurrency. Thus, we will explore different forms of concurrency control that can run in eBPF.

After the first implementation of a transaction manager, we will explore approaches that allow for inter-query parallelism (e.g., 2PL, OCC). eBPF has simple atomic primitives and latches (i.e., spinlocks) to implement basic mutual exclusion operations when multiple kernel threads are runnning. DBMSs use these operations to implement locking schemes to provide transaction isolation guarantees. However, eBPF has the added complexity of disallowing holding multiple spinlocks at once in order to avoid deadlocks. This makes methods like OCC difficult to implement due to the complex logic that must occur atomically at transaction completion. Transaction rollback with OCC also becomes more complex to perform deterministically, while locking makes it easier to express guarantees to eBPF's verifier.

After our initial transaction manager implementation, we will investigate the challenges of concurrent execution with multiple kernel threads processing transactions. One of the major challenges will be constructing a concurrency control scheme that can express parallelism and ACID properties without waiting. Locking schemes typically require either deadlock detection or deadlock avoidance. Deadlock detection could run in a scheduled eBPF program, but cycle detection algorithms may be too complex to pass the verifier. Instead, a deadlock avoidance scheme is more succinct. But, this still requires threads to wait according to a policy (e.g., wound-wait, wait-die) which is not possible in eBPF. The Redis protocol does not support interactive transactions, which makes them amenable to concurrency control methods found in deterministic database systems. We will explore such approaches like Very Lightweight Locking (VLL) [51]. We believe that a system with VLL and a no-wait policy is suitable to a user-bypass design while maintaining the benefits of concurrent transaction execution. Finally, we will explore if interactive transactions are possible in user-bypass, and what trade-offs are necessary to satisfy eBPF's constraints.

5.3 Logging and Checkpointing

In-memory DBMSs must write their database contents to persistent storage to ensure durability. These systems rely on two different methods to guarantee that they can restore their database contents after a restart: logging and checkpointing. DBMSs support both approaches, and we will explore the challenges of implementing each via user-bypass.

Logging requires the DBMS to maintain a ledger of all database modifications. Upon restart, the DBMS replays the log to reconstruct the database contents. The standard method to perform these tasks is ARIES [47]. Most DBMSs expose a knob that determines whether transactions commit synchronously or asynchronously. With asynchronous durability, the DBMS submits a transaction's changes to the component writing the log, but is allowed to acknowledge trans-

action completion to the client without guaranteeing that the changes are persistent on disk. Synchronous durability ensures that all contents of a transaction's changes have been sent to the OS via the `write()` system call, and in some cases also guarantees that the system also performed an `fsync()` to ensure OS write buffers are flushed.

Checkpointing is the process of writing the entire contents of the database to disk, thus preserving a consistent snapshot of the database at a single point in time. On restart, the DBMS reads the checkpoint file's contents to load the database contents into memory. The simplest approach to checkpointing is to block all transactions that are not read-only while flushing the database contents to disk. Once the checkpoint is complete, all transactions can resume. This approach limits the transaction performance of the system if there are not predictable periods of quiscence. DBMSs more commonly support fuzzy checkpointing, whereby all transactions continue to run during the checkpointing process, but the DBMS performs extra bookkeeping to guarantee that the checkpointing component sees a consistent snapshot of the database contents.

Logging and checkpointing components are the least amenable to applying user-bypass. This conflict stems from eBPF's inability to write to disk. I/O operations like writes can block for an indeterminate amount of time, which is not allowed during the execution of eBPF programs. For this reason, we propose building user-space components to handle disk writes. We will investigate how to synchronize the kernel-space and user-space components of the DBMS. The initial implementation will support asynchronous commit for transactions. Upon transaction completion, the user-bypass query executor will submit any database changes to user-space via a shared ringbuffer. The user-space component will read from this buffer, write the contents to disk, and ensure durability with `fsync()`. We will explore the possibility of synchronous commit, but as described in Section 5.2 it is not possible to make a eBPF program wait.

Similar to the user-space logging component, we will build a user-space checkpointing component. Techniques that rely on virtual memory's copy-on-write to create a snapshot of database contents by calling `fork()` on the DBMS's process will not work with a user-bypass system because the database is stored in kernel memory [40]. Instead, we will implement fuzzy checkpointing by reading the eBPF maps storing the database contents from user-space, and investigate what logic must occur in the user-bypass query executors to maintain a consistent snapshot while a checkpoint occurs [52]. Like many kernel data structures, eBPF maps support the read-copy-update (RCU) synchronization method to present consistent views of changing data structures. We believe that the DBMS can exploit these semantics to support checkpointing.

Chapter 6

Thesis Timeline

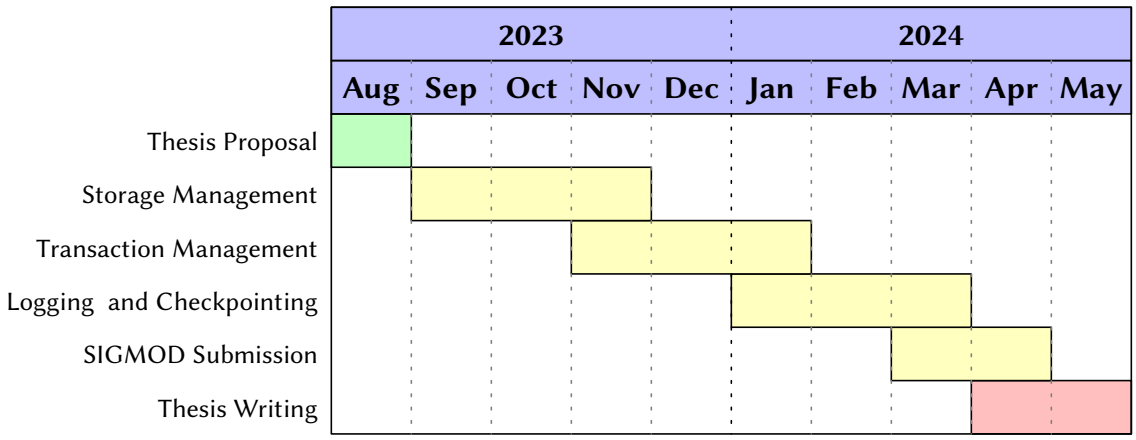


Figure 6.1: Expected timeline to complete thesis.

Bibliography

[1] ENA Linux Driver Best Practices and Performance Optimization Guide. https://github.com/amzn/amzn-drivers/blob/master/kernel/linux/ena/ENA_Linux_Best_Practices.rst. 4.2

[2] GitHub - cmu-db/benchbase: Multi-DBMS SQL Benchmarking Framework via JDBC. https://github.com/cmu-db/benchbase. 3.3.1, 4.2.1

[3] GitHub - cilium/cilium: eBPF-based Networking, Security, and Observability. https://github.com/cilium/cilium, . 2.2

[4] GitHub - facebookincubator/katran: A high performance layer 4 load balancer. https://github.com/facebookincubator/katran, . 2.2

[5] GitHub - microsoft/ebpf-for-windows: eBPF implementation that runs on top of Windows. https://github.com/microsoft/ebpf-for-windows, . 2.2

[6] Intel Data Plane Development Kit (DPDK). https://www.dpdk.org. 2.1

[7] Amazon EC2 C6i Instances - Amazon Web Services. https://aws.amazon.com/ec2/instance-types/c6i/. 4.2

[8] F-Stack | High Performance Network Framework Based On DPDK. http://www.f-stack.org. 2.1

[9] Github - haproxy/haproxy: Haproxy load balancer's development branch (mirror of git.haproxy.org). https://github.com/haproxy/haproxy/. 4.1

[10] Github - mariadb-corporation/maxscale: An intelligent database proxy. https://github.com/mariadb-corporation/MaxScale. 4.1

[11] nginx. https://nginx.org/en/. 4.1

[12] Github - yandex/odyssey: Scalable postgresql connection pooler. https://github.com/yandex/odyssey. 4.1

[13] Self-driving database | autonomous database oracle 19c. https://oracle.com/database/autonomous-database/. 3.1

[14] PgBouncer - lightweight connection pooler for PostgreSQL. https://www.pgbouncer.org. 4.1

[15] pgpool Wiki. https://www.pgpool.net/. 4.1

[16] PostgreSQL: Documentation: 14: EXPLAIN. https://www.postgresql.org/docs/current/sql-explain.html. 3.2.2

[17] Proxysql - a high performance open source mysql proxy. https://proxysql.com. 4.1

[18] Amazon RDS Proxy | Highly Available Database Proxy | Amazon Web Services. https://aws.amazon.com/rds/proxy/. 4.1

[19] Seastar. https://seastar.io. 2.1

[20] socket(2) - Linux manual page. https://www.man7.org/linux/man-pages/man2/socket.2.html. 2.1

[21] tc(8) - Linux manual page. https://man7.org/linux/man-pages/man8/tc.8.html. 2.1

[22] Introduction to the Storage Performance Development Kit (SPDK). https://www.intel.com/content/www/us/en/developer/articles/tool/introduction-to-the-storage-performance-development-kit-spdk.html, 2015. 2.1

[23] David Ahern. Can the linux networking stack be used with very high speed applications? https://lpc.events/event/16/contributions/1345/, September 2022. 2.1

[24] Mohammad Alomari, Michael J. Cahill, Alan D. Fekete, and Uwe Röhm. The cost of serializability on platforms that use snapshot isolation. In *ICDE*, pages 576–585. IEEE Computer Society, 2008. 3.3.1, 4.2.1

[25] Brian N. Bershad, Stefan Savage, Przemyslaw Pardyak, Emin Gün Sirer, Marc E. Fiuczynski, David Becker, Craig Chambers, and Susan J. Eggers. Extensibility, safety and performance in the SPIN operating system. In *SOSP*, pages 267–284. ACM, 1995. 1

[26] Daniel Borkmann and John Fastabend. Combining ktls and bpf for introspection and policy enforcement. November 2018. 2.1

[27] Michael J. Cahill, Uwe Röhm, and Alan D. Fekete. Serializable isolation for snapshot databases. In *SIGMOD Conference*, pages 729–738. ACM, 2008. 3.3.1, 4.2.1

[28] Qizhe Cai, Shubham Chaudhary, Midhul Vuppalapati, Jaehyun Hwang, and Rachit Agarwal. Understanding host network stack overheads. In *SIGCOMM*, pages 65–77. ACM, 2021. 2.1, 4.1

[29] Brian E. Carpenter and Scott W. Brim. Middleboxes: Taxonomy and issues. *RFC*, 3234: 1–27, 2002. 4.1

[30] Elizabeth Christensen. Postgres at scale: Running multiple pgbouncers. https://www.crunchydata.com/blog/postgres-at-scale-running-multiple-pgbouncers, November 2022. 4.1

[31] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking cloud serving systems with YCSB. In *SoCC*, pages 143–154. ACM, 2010. 3.3.1, 4.2.1

[32] Sudipto Das, Miroslav Grbic, Igor Ilic, Isidora Jovandic, Andrija Jovanovic, Vivek R Narasayya, Miodrag Radulovic, Maja Stikic, Gaoxiang Xu, and Surajit Chaudhuri. Automatically indexing millions of databases in microsoft azure sql database. In *SIGMOD '19*, pages 666–679, 2019. 3.1

[33] Djellel Eddine Difallah, Andrew Pavlo, Carlo Curino, and Philippe Cudré-Mauroux. Oltpbench: An extensible testbed for benchmarking relational databases. *Proc. VLDB Endow.*, 7(4):277–288, 2013. 3.3.1, 4.2.1

[34] Rodrigo Fonseca, George Porter, Randy H. Katz, and Scott Shenker. X-trace: A pervasive network tracing framework. In *4th USENIX Symposium on Networked Systems Design & Implementation (NSDI 07)*, April 2007. 3.2.1

[35] CMU Database Group. ScyllaDB: No-Compromise Performance (Avi Kivity). https://

youtu.be/0S6i9BmuF8U?t=2586, 2020. 2.1

[36] CMU Database Group. Yellowbrick: An Elastic Data Warehouse on Kubernetes (Mark Cusack). https://youtu.be/uHMcVDNkHi4, 2022. 2.1

[37] Yongjun He, Jiacheng Lu, and Tianzheng Wang. Corobase: Coroutine-oriented main-memory database engine. *Proc. VLDB Endow.*, 14(3):431–444, 2020. 1

[38] Toke Høiland-Jørgensen, Jesper Dangaard Brouer, Daniel Borkmann, John Fastabend, Tom Herbert, David Ahern, and David Miller. The express data path: fast programmable packet processing in the operating system kernel. In *CoNEXT*, pages 54–66. ACM, 2018. 2.1

[39] Eunyoung Jeong, Shinae Woo, Muhammad Asim Jamshed, Haewon Jeong, Sunghwan Ihm, Dongsu Han, and KyoungSoo Park. mtcp: a highly scalable user-level TCP stack for multicore systems. In *NSDI*, pages 489–502. USENIX Association, 2014. 2.1

[40] Alfons Kemper and Thomas Neumann. Hyper: A hybrid oltp&olap main memory database system based on virtual memory snapshots. In *ICDE*, pages 195–206. IEEE Computer Society, 2011. 5.3

[41] Tianyu Li, Matthew Butrovich, Amadou Ngom, Wes McKinney, and Andrew Pavlo. Mainlining databases: Supporting fast transactional workloads on universal columnar data file formats. volume 14, pages 534–546. VLDB Endowment, 2021. 3.3, 5.1

[42] Lin Ma, Dana Van Aken, Ahmed Hefny, Gustavo Mezerhane, Andrew Pavlo, and Geoffrey J. Gordon. Query-based workload forecasting for self-driving database management systems. In *SIGMOD*, pages 631–645, 2018. 3.1

[43] Lin Ma, William Zhang, Jie Jiao, Wuwen Wang, Matthew Butrovich, Wan Shen Lim, Prashanth Menon, and Andrew Pavlo. Mb2: Decomposed behavior modeling for self-driving database management systems. In *SIGMOD*, pages 1248–1261, 2021. 3.1, 3.3

[44] Ryan Marcus and Olga Papaemmanouil. Plan-structured deep neural network models for query performance prediction. *Proc. VLDB Endow.*, 12(11):1733–1746, 2019. 3.1, 3.2.1, 3.2.2

[45] Paul E. McKenney, Joel Fernandes, Silas Boyd-Wickizer, and Jonathan Walpole. RCU usage in the linux kernel: Eighteen years later. *ACM SIGOPS Oper. Syst. Rev.*, 54(1):47–63, 2020. 3.3.2

[46] John McNamara, Ian Stokes, Luca Boccassi, and Kevin Traynor. API/ABI Stability and LTS: Current state and Future. https://www.dpdk.org/event/dpdk-userspace-dublin-2017/, 2017. 2.1

[47] C. Mohan, Don Haderle, Bruce G. Lindsay, Hamid Pirahesh, and Peter M. Schwarz. ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Trans. Database Syst.*, 17(1):94–162, 1992. 5.3

[48] Thomas Neumann, Tobias Mühlbauer, and Alfons Kemper. Fast serializable multi-version concurrency control for main-memory database systems. In *SIGMOD*, pages 677–689, 2015. 3.3

[49] Andrew Pavlo, Gustavo Angulo, Joy Arulraj, Haibin Lin, Jiexi Lin, Lin Ma, Prashanth Menon, Todd Mowry, Matthew Perron, et al. Self-driving database management systems. In *CIDR*, 2017. 3.1

[50] Andrew Pavlo, Matthew Butrovich, Ananya Joshi, Lin Ma, Prashanth Menon, Dana Van Aken, Lisa Lee, and Ruslan Salakhutdinov. External vs. internal: An essay on machine

learning agents for autonomous database management systems. *IEEE Data Engineering Bulletin*, pages 32–46, June 2019. 3.1

[51] Kun Ren, Alexander Thomson, and Daniel J. Abadi. VLL: a lock manager redesign for main memory database systems. *VLDB J.*, 24(5):681–705, 2015. 5.2

[52] Kun Ren, Thaddeus Diamond, Daniel J. Abadi, and Alexander Thomson. Low-overhead asynchronous checkpointing in main-memory database systems. In *SIGMOD Conference*, pages 1539–1551. ACM, 2016. 5.3

[53] Felix Martin Schuhknecht, Jens Dittrich, and Ankur Sharma. RUMA has it: Rewired user-space memory access is possible! *Proc. VLDB Endow.*, 9(10):768–779, 2016. 1

[54] Margo I. Seltzer, Yasuhiro Endo, Christopher Small, and Keith A. Smith. Dealing with disaster: Surviving misbehaved kernel extensions. In *OSDI*, pages 213–227. ACM, 1996. 1

[55] Slava Shwartsman and Drew Gallatin. Kernel tls and tls hardware offload. `https://papers.freebsd.org/2019/eurobsdcon/shwartsman_gallatin-kernel_tls_harware_offload/`, September 2019. 2.1

[56] Alexei Starovoitov. LKML: Alexei Starovoitov [PATCH net-next] extended BPF. `https://lkml.org/lkml/2013/9/30/627`, 2013. 1, 2.2

[57] Alexei Starovoitov. BPF at Facebook. `https://kernel-recipes.org/en/2019/talks/bpf-at-facebook/`, 2019. 2.2

[58] Michael Stonebraker. Operating system support for database management. *Commun. ACM*, 24(7):412–418, 1981. 1

[59] The Transaction Processing Council. TPC-C Benchmark (Revision 5.9.0). `http://www.tpc.org/tpcc/spec/tpcc_current.pdf`, June 2007. 3.3.1, 4.2.1

[60] William Tu, Yi-Hung Wei, Gianni Antichi, and Ben Pfaff. revisiting the open vswitch dataplane ten years later. In *SIGCOMM*, pages 245–257. ACM, 2021. 1

[61] Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. Automatic database management system tuning through large-scale machine learning. In *SIGMOD*, pages 1009–1024, 2017. 3.1

[62] Antoni Wolski. TATP Benchmark. `http://tatpbenchmark.sourceforge.net`. 3.3.1, 4.2.1

[63] Yingjun Wu, Joy Arulraj, Jiexi Lin, Ran Xian, and Andrew Pavlo. An empirical evaluation of in-memory multi-version concurrency control. *Proc. VLDB Endow.*, 10(7):781–792, 2017. 5.2

[64] Yuhong Zhong, Haoyu Li, Yu Jian Wu, Ioannis Zarkadas, Jeffrey Tao, Evan Mesterhazy, Michael Makris, Junfeng Yang, Amy Tai, Ryan Stutsman, and Asaf Cidon. XRP: in-kernel storage functions with ebpf. In *OSDI*, pages 375–393. USENIX Association, 2022. 1